

MODULE 12 · DOMAIN 4 - PROMPT ENGINEERING & STRUCTURED OUTPUT

Prompting for Judgement

Explicit criteria over emphasis, when examples earn their place, and why one prompt doing two jobs does both badly.

DOMAIN 4 of 5 EXAM WEIGHT 20% MODULE 12 of 20

When a prompt underperforms, the model is rarely failing to **try**. It is failing to know what you meant.

Before you optimise anything

- ◆ **Define what success looks like**, in terms concrete enough that two people would agree.
- ◆ **Build a way to test against it**. A handful of real cases with known answers.
- ◆ **Write a first draft**, and improve it against the tests.
- ◆ **Without those three you are optimising blind**, and every change feels like an improvement.

ON THE EXAM

Prompt engineering is far cheaper and faster than fine-tuning. That is only true if you can tell whether it worked.

The technique ladder, in order

TECHNIQUE	REACH FOR IT WHEN
Be clear and direct	Always. This is the first and largest lever.
Examples (multishot)	Format is inconsistent, or the task needs judgement
Let Claude think	The task has reasoning steps worth doing explicitly
Structure the prompt	Instructions and data are getting confused
Role or system prompt	Tone and domain framing matter
Prefill the response	You need to control how the output starts
Chain complex prompts	One prompt is doing too much and doing it unreliably

ON THE EXAM

Anthropic orders these from broadly effective to specialised. Work down the list; do not start at the bottom.

Emphasis is not a technique. **Be thorough**
in capitals is still **be thorough**.

A quality bar, or a checklist?

VAGUE

"Review this code carefully"

- Careful means something different each run
- Coverage varies with what catches its attention
- You cannot tell a miss from a judgement call
- Adding "very" changes nothing

EXPLICIT

Named categories, one at a time

- "Check for: SQL injection, missing auth, unhandled errors"
- Coverage is the same every run
- A miss is visible because the category is named
- You can add a category when you find a gap

THE TIEBREAKER

Explicit criteria do not make the model cleverer. They make its coverage repeatable, which is what you actually needed.

Writing criteria that generalise

- ◆ **Describe the property, not the instance.** "Amounts above the approval limit", not "the £600 case".
- ◆ **State what is out of scope** as explicitly as what is in it.
- ◆ **Give the reason for a constraint.** Claude follows a rule better when it knows what the rule protects.
- ◆ **A rule written from one bad example fixes one bad example.** The next one will be shaped differently.

ON THE EXAM

Most effective on the exam usually means the answer that generalises, not the one that handles the case in the stem.

Examples, or more instructions?

FEW-SHOT

Show it

- The output format is inconsistent
- The task is a judgement call, hard to define
- Edge cases need a demonstrated resolution
- Tone and register matter

EXPLICIT CRITERIA

Tell it

- The rule is a rule: include this, exclude that
- Coverage needs to be complete and checkable
- The boundary is definable in words
- You need to audit what it was told

THE TIEBREAKER

Examples are for things easier to demonstrate than to define. Criteria are for things easier to define than to demonstrate.

Using examples well

- ◆ **Three to five is usually enough**, and diversity matters more than quantity.
- ◆ **Include a hard case** and show how it resolves. That is where the teaching happens.
- ◆ **Examples that all look alike teach a narrow rule** the model then applies too confidently.
- ◆ **Examples are input tokens on every call**. They are not free, and a long set is a real recurring cost.

ON THE EXAM

An example set that contains no difficult case teaches the model that there are no difficult cases.

One prompt doing two jobs will do **both** worse. Attention is finite, and so is instruction-following.

Recognising an overloaded prompt

- ◆ **It does one half well and the other half thinly**, and which half varies.
- ◆ **Strengthening the weak half weakens the strong half**. That trade is the diagnosis.
- ◆ **Findings from one category crowd out another** rather than adding to it.
- ◆ **The fix is two calls**, each with a narrow brief, with results combined afterwards.

ON THE EXAM

"We improved X and Y got worse" is the clearest signal in the whole domain. It means one prompt, two jobs.

Chaining: what it costs and buys

- ◆ **Costs: more calls, more latency, more orchestration.** Two prompts are not free.
- ◆ **Buys: each call has one job,** so each can be tuned, tested and measured separately.
- ◆ **Buys: you can see which stage failed,** instead of guessing which half of a prompt went wrong.
- ◆ **Worth it when reliability matters more than latency,** which on this exam is most of the time.

ON THE EXAM

This is module 2's prompt chaining, arriving from the prompt engineering side rather than the architecture side.

Filter in the prompt, or downstream?

IN THE PROMPT

"Only report issues you are confident about"

- The model decides what counts as confident
- That threshold moves between runs
- Suppressed findings are invisible to you
- You cannot tune it without rewriting

DOWNSTREAM

Report everything, scored

- Every finding is visible and recorded
- The threshold is a number in your code
- Tune it against real outcomes
- Low confidence routes to review

THE TIEBREAKER

Asking the model to filter hides what it discarded. Asking it to score lets you decide, and change your mind later.

Handling ambiguity

- ◆ **State the assumption and continue** when the ambiguity is minor and the assumption is checkable.
- ◆ **Ask** when the choice materially changes the output and getting it wrong wastes the work.
- ◆ **Do not ask three questions before starting.** That is an interrogation, not a clarification.
- ◆ **An explicit assumption is reviewable.** A silent one is a bug waiting to be discovered.

ON THE EXAM

The worst option is always the one that proceeds on an unstated assumption. Both stating and asking beat guessing quietly.

Reviewing in passes

- ◆ **A separate pass per concern** beats one pass covering everything, for the reasons two slides ago.
- ◆ **Ask Claude to verify its own work explicitly** before declaring a task complete.
- ◆ **Self-verification inside the same context is weak.** It agrees with itself more than it should.
- ◆ **A fresh pass, or a deterministic check, is stronger.** Module 11's fresh-session review is this idea.

ON THE EXAM

Verification strength runs: a deterministic check, then a fresh context, then self-review, then nothing.

Troubleshooting map

SYMPTOM	REACH FOR
Output format keeps varying	Examples, or prefill the start of the response
Coverage varies between runs	Named categories rather than a quality adjective
Reasoning is wrong on hard cases	Let it think, step by step, before answering
Instructions and data get confused	Structure the prompt so they are clearly separated
Improving one thing degrades another	Split into two prompts
Findings you never see	Score and filter downstream, not in the prompt

ON THE EXAM

Match the symptom to the technique. Do not reach for the technique you happen to like.

Currency check: two techniques have softened

TOPIC	EXAM ANSWER	CURRENT REALITY
XML tags	A recommended way to structure prompts	Still useful, but modern models handle structure well without them.
Role prompting	Set a role for accuracy and tone	Heavy role prompting is less necessary. Start with explicit instructions.
The cost of technique	Not emphasised	Examples and chaining consume tokens. Use them when they earn it.

ON THE EXAM

Neither technique became wrong. Both became less load-bearing than the guide-era framing suggests.

Module 12 in five lines

- ◆ When a prompt underperforms, be more specific rather than more forceful.
- ◆ Named categories fix coverage. Adjectives like careful and thorough do not.
- ◆ Examples for what is easier to show than to define. Criteria for the reverse.
- ◆ One prompt, two jobs, both worse. "We improved X and Y got worse" means split it.
- ◆ Score and filter downstream. A model asked to filter hides what it discarded.

ON THE EXAM

Six scenarios next. Two of them offer a stronger prompt as the tempting wrong answer.

A contract review prompt says: "Review this contract carefully and thoroughly. Identify any risky or unusual terms. Be extremely diligent."

Across a hundred contracts, the model finds unusual payment terms reliably, mentions liability caps sometimes, and has never once flagged an automatic renewal clause.

The team has strengthened the wording twice already.

What is the most effective change?

- ☐ A Strengthen the wording further and add that renewal clauses are especially important
- ☐ B Replace the adjectives with a named list of clause categories to check, including automatic renewal
- ☐ C Add three example contracts with their correct reviews
- ☐ D Use a larger model, since contract review is a demanding task

◆ PAUSE AND DECIDE

What is the most effective change?

- A

Strengthen further

Two attempts already failed. It also patches one category rather than fixing how coverage is defined.
- B

Named categories

Turns a variable sweep into a fixed checklist. A category can be added the moment a gap is found.
- C

Example reviews

Would help with format and judgement. It cannot guarantee coverage of a category the examples happen not to feature.
- D

Larger model

The model is not failing to understand contracts. It has never been told renewal clauses are in scope.

THE SIGNAL

"Never once" is the tell. A capability problem produces sometimes. A scope problem produces never.

A review prompt asks Claude to both check code against a twelve-point style guide and identify security vulnerabilities, in a single call.

When the team emphasised security, style findings dropped noticeably. When they later re-emphasised style, security findings dropped. Total findings per review stayed roughly constant.

Both concerns matter equally to them.

What should they do?

- ☐ A Emphasise both equally and accept the current balance
- ☐ B Run two calls, one for style and one for security, and merge the results
- ☐ C Ask for security first and style second, so security is done while attention is fresh
- ☐ D Increase max_tokens so there is room for findings from both categories

◆ PAUSE AND DECIDE

What should they do?

- A

 Balance the emphasis
Accepts the trade instead of removing it. Both categories stay mediocre by agreement.
- B

 Two calls, merged
Each call has one job, so neither competes with the other. Tunable and measurable separately.
- C

 Order them
Reorders the competition without ending it. Whichever goes second still gets the remainder.
- D

 More output tokens
Total findings stayed constant while the mix shifted. Nothing was being truncated.

THE SIGNAL

"Total findings stayed roughly constant" is the planted detail. That rules out truncation and confirms competition.

A support triage prompt classifies incoming messages into six categories. The categories are clearly defined and mutually exclusive.

Accuracy is 91%. The errors are concentrated in genuinely borderline messages: a billing complaint that is also a cancellation threat, a bug report phrased as a feature request.

The team wants to improve accuracy on those borderline cases.

What is most likely to help?

- ☐ A Add few-shot examples of borderline messages with their correct classifications and brief reasoning
- ☐ B Rewrite the six category definitions to be even more precise
- ☐ C Add a seventh category called "ambiguous"
- ☐ D Instruct the model to be more careful with messages that could fit two categories

◆ PAUSE AND DECIDE

What is most likely to help?

- A

 Borderline examples with reasoning
The boundary is a judgement call, and judgement is easier to demonstrate than to define.
- B

 More precise definitions
The definitions are already clear. The difficulty is which of two clear categories applies.
- C

 An "ambiguous" bucket
Moves hard cases into a pile nobody has a process for. It hides the problem as a category.
- D

 Be more careful
An adjective. It does not tell the model how to resolve a tie it can already see.

THE SIGNAL

Pair this with scenario 1. There, criteria beat examples. Here, examples beat criteria, because the boundary is judgement, not scope.

A document analysis prompt ends with: "Only report findings you are highly confident about, to avoid overwhelming reviewers."

An audit finds that several material issues were present in documents where the model reported nothing. There is no record of what it considered and discarded.

Reviewers say they would rather see uncertain findings than miss real ones.

What should change?

- ☐ A Lower the confidence bar in the prompt to "reasonably confident"
- ☐ B Report all findings with a confidence score, and filter in the pipeline against a tuned threshold
- ☐ C Remove the confidence instruction so everything is reported
- ☐ D Ask the model to explain why it discarded low-confidence findings

◆ PAUSE AND DECIDE

What should change?

- A

 Lower the bar in words
Moves an invisible threshold to a slightly different invisible place. Still no record of what was dropped.
- B

 Score, then filter downstream
Everything is recorded, the threshold is a number you can tune, and low confidence can route to review.
- C

 Report everything
Solves the misses by creating the overwhelm the instruction existed to prevent.
- D

 Explain the discards
Better than nothing, and it asks the model to report reliably on a decision it already made silently.

THE SIGNAL

"No record of what it considered and discarded" is the sentence to notice. Any answer leaving the filter in the model keeps that blind spot.

An engineer asks an agent to "summarise the quarterly results for the board". The agent produces a two-page narrative summary.

The engineer wanted half a page of bullet points with figures. They ask again, more specifically, and get what they wanted.

This pattern repeats across many different open-ended requests from many different users.

What is the best general fix?

- ☐ A Add a default output format to the system prompt: half a page, bullets, figures included
- ☐ B Have the agent state its assumptions about scope and format before producing long output
- ☐ C Have the agent ask three clarifying questions before every request
- ☐ D Add few-shot examples of good summaries

◆ PAUSE AND DECIDE

What is the best general fix?

- A

A default format

Fixes summaries. The stem says many different requests from many users, and a board summary default suits none of the others.
- B

State assumptions first

Generalises across request types, costs a line, and lets the user correct before the work is done.
- C

Three questions every time

Interrogation. Most requests are not ambiguous, and the friction lands on all of them.
- D

Example summaries

Same limitation as A. It teaches one output shape for one request type.

THE SIGNAL

"Many different open-ended requests from many different users" is the constraint. It rules out every fix specific to summaries.

A prompt asks Claude to extract every clause in a policy document that imposes an obligation on the customer, then rate each obligation's severity on a five-point scale. Extraction recall is high when severity ratings are omitted. When ratings are requested, several obligations go missing entirely, and the ones extracted are rated consistently. The team needs both.

What is the most effective approach?

- ☐ A Ask for extraction and rating in two separate calls, feeding the extracted clauses into the second
- ☐ B Ask the model to extract everything first, then rate, within the same response
- ☐ C Emphasise that no obligation should be missed, regardless of rating effort
- ☐ D Drop the severity scale to three points to reduce the rating burden

◆ PAUSE AND DECIDE

What is the most effective approach?

- A** Two calls, chained
Recall gets a call to itself, then rating gets a call to itself. The pairing is done in code, not in attention.
- B** Sequence within one response
The strongest distractor. Better than interleaving, and both jobs still share one attention budget.
- C** Emphasise completeness
Swaps which half suffers. Emphasis redistributes the trade rather than removing it.
- D** A shorter scale
Reduces the rating burden slightly. The competition remains, just smaller.

THE SIGNAL

The A/B split is the useful one. Ordering within a response helps a little; separate calls remove the competition entirely.